Assignment 1: Fine-Tune, Sanitize, Extract

Entry Number: `2025JVL2045`

1 Part A: Where Do You Put the Adapters?

1.1 Experimental setup

We fine-tuned `Qwen/Qwen2.5-1.5B-Instruct` to map each `source_text` field to its corresponding `target_text` field. Each prompt used the fixed instruction *"Sanitize the following text:"*. The training loss was applied only to completion tokens; prompt tokens were masked from the loss.

Every configuration uses the supplied 20,000-record training split and the supplied 2,000-record validation split in their original order. The seed is 42. The epoch count, learning-rate schedule, effective batch size, maximum sequence length, and LoRA dropout were taken to be identical across A1–A6.Then recorded trainable parameters using `model.print_trainable_parameters()` and peak GPU memory using `torch.cuda.max_memory_allocated()`.

Table 1: Required Part A configurations.

Run	Method	Target modules	r	α	Quantization
A1	LoRA	`q_proj, v_proj`	8	16	none
A2	LoRA	`q_proj, k_proj, v_proj, o_proj`	8	16	none
A3	LoRA	attention modules + `gate_proj, up_proj, down_proj`	8	16	none
A4	LoRA	same as A3	32	64	none
A5	LoRA	same as A3	32	16	none
A6	QLoRA	same as A3	32	64	4-bit

1.2 Validation metrics

Entity F1 is computed from generated validation outputs using the Part B evaluation procedure. Leak rate is the fraction of gold PII values that remain verbatim in the generated sanitized text, after case and whitespace normalization. The source file for Table 2 is `results/partA_sweep.csv`. The A6 held-out validation values were computed from 2,000 saved outputs and are recorded in `results/partA_A6_checkpoint7000_fast_metrics.json`. A6 validation loss could not be logged (an CUDA limit error while A6 run).

Table 2: Part A three-epoch sweep results from `results/partA_sweep.csv`.

Run	Trainable %	Peak GPU (GB)	Time (s)	Final train loss	Entity F1	Leak rate
A1	0.071	4.593	7286	0.0157	0.9299	0.0402
A2	0.141	4.716	7964	0.0119	0.9381	0.0392
A3	0.595	5.214	10513	0.0081	0.9481	0.0372
A4	2.336	5.419	10787	0.0083	0.9538	0.0383
A5	2.336	5.419	10883	0.0080	0.9522	0.0380
A6	2.336	3.473	13231	–	0.9550	0.0369

† A6 Entity F1 and leak rate were recomputed from 2,000 saved validation outputs.

1.3 Results and analysis

For each change in configuration, we compare the increase in entity F1 against the additional trainable-parameter percentage, peak GPU memory, and training time. The reported best axis is the one with the largest F1 gain per additional resource consumed.

The performance curve begins to flatten at rank 32: A4 and A5 achieve nearly the same F1 (0.9538 and 0.9522) despite different α values. A6 gives the highest recorded F1 (0.9550) and lowest leak rate (0.0369). With the same trainable percentage as A4, it reduces peak GPU memory from 5.419 to 3.473 GB, at the cost of a longer training time (13,231 s). Its validation loss was not logged (CUDA limit error occu) and is therefore left unavailable in Table 2.

QLoRA is compared directly with A4 because both use rank 32, $\alpha = 64$, and the same target modules. The results show:

- an Entity F1 increase of 0.0011 (from 0.9538 to 0.9550);
- peak GPU memory savings of 1.946 GB (from 5.419 GB to 3.473 GB); and
- a 35.9% reduction in peak GPU memory.

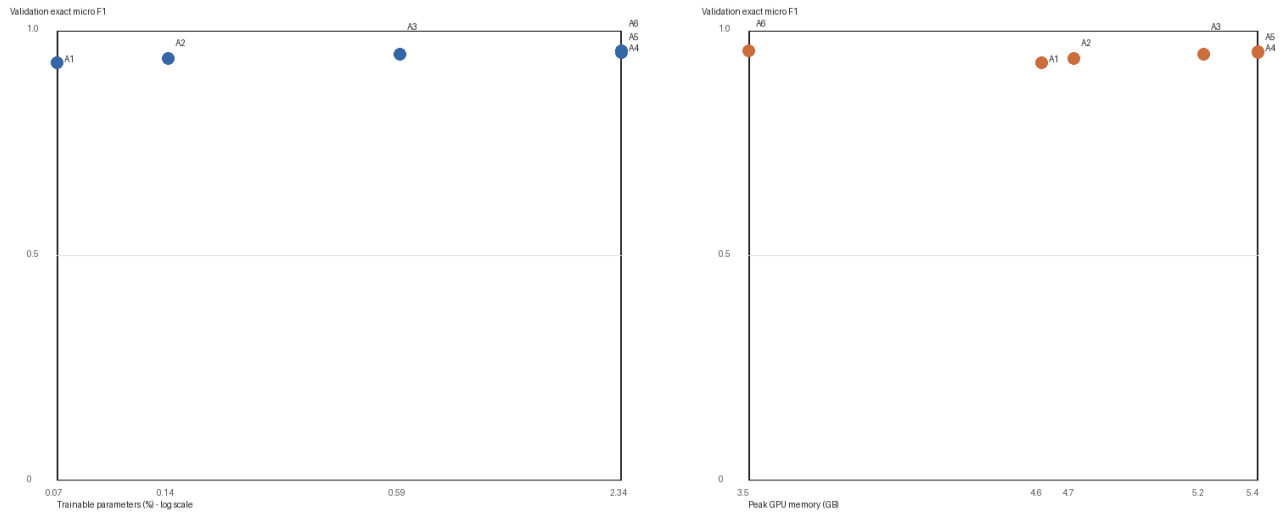


Figure 1: Part A PEFT efficiency comparison. Left: validation Entity F1 against trainable-parameter percentage on a log-scale x-axis. Right: peak GPU memory against validation Entity F1.

1.4 Part A figure

2 Part B: How Good Is the Sanitizer, Really?

2.1 Checkpoint selection and protocol

We evaluated the A4 LoRA adapter on the supplied held-out `partB_test.jsonl` (2,000 records) and `partB_ood_test.jsonl` (1,000 records), without changing their order or contents. A4 was selected as the established full-precision sanitizer for Part B: it achieves 95.38% validation exact F1 with the rank-32, $\alpha = 64$ configuration. Thus, A4 provides a reproducible sanitizer for the Part B evaluation.

For all model generations, decoding was deterministic (`do_sample=False`, one beam, seed 42). The prompt was `Sanitize the following text:` followed by the source text. The complete implementation is in `src/sanitize_eval.py`; generated outputs and aggregate reports are stored in `results/`.

2.2 B.1: Span recovery from generated text

The sanitizer produces masked text rather than character offsets. We therefore recover predicted spans by aligning `source_text` with the generated output using `difflib.SequenceMatcher` with `autojunk=False`. Each generated placeholder of the form `[LABEL]` is temporarily replaced by a unique sentinel character before alignment. Exact common prefixes and suffixes are locked first, then the remaining regions are aligned.

A changed source interval containing exactly one placeholder becomes one predicted span $(start, end, label)$. Numeric suffixes used for repeated masks, such as `[EMAIL_2]`, are removed only for evaluation, yielding the label `EMAIL`. A changed region containing multiple placeholders is not converted into spans and is counted as an ambiguous alignment region. Zero-width insertions similarly produce no span. Literal deletions and rewrites still contribute to over-masking.

This procedure can fail when the model rewrites surrounding text, when source text contains repeated substrings, when a single edit contains multiple masks, or when literal placeholder-like text appears in the source. Therefore, well-formedness is reported separately: an output is well formed only when it has no unmatched brackets, chat-control tokens, code fences, or ambiguous multi-placeholder alignment regions.

2.3 B.2: Metrics

Let G and P denote gold and predicted entities, respectively. Exact matching requires identical start offset, end offset, and label. Partial matching requires the same label and intersection-over-union IoU ≥ 0.5. Both matching procedures enforce one-to-one matching: a predicted or gold entity may participate in at most one pair. For IoU evaluation, the implementation obtains the maximum-total-IoU one-to-one assignment among all valid same-label pairs.

We report micro and macro precision, recall, and F1. Macro scores are the unweighted average over the union of gold and predicted classes, with undefined values assigned zero. Leak rate is the fraction of gold entity occurrences whose value remains in the output after lowercasing and collapsing whitespace. Over-masking rate is the number of changed non-PII source characters divided by all non-PII source characters; character unions prevent double counting.

2.4 In-distribution results and baseline comparison

Table 3 reports every required metric for the fine-tuned sanitizer and both non-fine-tuned baselines. The values come from `results/partB_metrics.json`, `results/partB_regex_metrics.json`, and `results/partB_presidio_metrics.json` for the in-distribution set, with the corresponding `_ood.json` files for OOD evaluation. The consolidated machine-readable table is `results/partB_results_table.csv`.

Table 3: Part B results. All values are percentages. μ denotes micro averaging and M denotes macro averaging.

		Exact span and label match						IoU ≥ 0.5 and label match								
System	Set	μP	μR	$\mu F1$	MP	MR	$MF1$	μP	μR	$\mu F1$	MP	MR	$MF1$	Leak	Over-mask	Well-formed
A4 LoRA	ID	94.49	95.23	94.86	53.72	54.13	53.90	95.08	95.82	95.45	54.12	54.54	54.31	3.81	0.581	74.85
Regex	ID	58.75	9.35	16.13	9.01	7.68	8.06	60.62	9.65	16.65	9.22	7.96	8.29	84.88	0.201	100.00
Presidio	ID	31.31	19.58	24.09	11.81	12.16	11.37	32.55	20.35	25.05	12.18	12.76	11.79	47.16	1.785	100.00
A4 LoRA	OOD	22.34	14.95	17.91	10.97	8.75	8.07	22.90	15.32	18.36	12.31	9.02	8.43	35.34	0.098	99.50
Regex	OOD	59.21	8.71	15.18	5.44	6.24	5.72	67.15	9.87	17.22	6.38	7.18	6.66	85.27	0.030	100.00
Presidio	OOD	15.80	9.81	12.11	6.21	6.73	5.30	17.53	10.89	13.43	6.82	7.42	5.93	41.40	1.038	100.00

On the in-distribution set, A4 achieves 94.86% exact micro F1 and 95.45% IoU micro F1. It has the lowest leak rate (3.81%), substantially below Presidio (47.16%) and the regex baseline (84.88%). The lower macro F1 of 53.90% relative to micro F1 reflects that some infrequent classes are considerably harder than the frequent classes.

The regex baseline was implemented from scratch and masks email addresses, telephone numbers, IP addresses, credit-card-like numbers, dates, and URLs. It is precise when its patterns apply, but has very low recall because it cannot identify context-dependent PII classes such as names, usernames, identity cards, and passwords. Presidio uses its default recognizers. Its coarse entity categories are mapped with a fixed mapping (for example, `EMAIL_ADDRESS`→`EMAIL` and `DATE_TIME`→`DATE`); no gold-dependent remapping was used. This explains part of its lower score on this fine-grained dataset.

2.5 Per-class performance

Table 4 reports performance for the ten most frequent in-distribution PII classes. The source is `results/partB_top10_classes.csv`. All frequent classes attain strong exact F1, with `LASTNAME1` being the most difficult (87.40%), consistent with ambiguity in names and local alignment boundaries.

Table 4: A4 LoRA performance on the ten most frequent in-distribution classes. All P/R/F1 values are percentages.

Class	Support	Exact			IoU ≥ 0.5		
		P	R	F1	P	R	F1
TIME	945	96.48	95.77	96.12	96.91	96.19	96.55
USERNAME	760	95.54	95.92	95.73	95.54	95.92	95.73
SOCIALNUMBER	668	97.72	96.41	97.06	98.33	97.01	97.66
EMAIL	662	97.15	97.73	97.44	98.20	98.79	98.49
IDCARD	651	93.44	94.01	93.72	93.89	94.47	94.18
DRIVERLICENSE	643	96.98	95.02	95.99	96.98	95.02	95.99
LASTNAME1	623	85.78	89.09	87.40	86.24	89.57	87.87
BOD	553	96.24	97.11	96.67	97.31	98.19	97.75
PASSPORT	552	92.03	96.20	94.07	92.55	96.74	94.60
SEX	529	97.38	98.49	97.93	97.94	99.05	98.50

2.6 Error analysis

For A4 on the in-distribution set, the error taxonomy in `results/partB_error_taxonomy.csv` records 155 missed entities, 142 boundary errors, 313 label errors, 709 unmatched predicted spans, and 15 ambiguous alignment regions. There were no empty outputs. The low well-formed rate of 74.85% is therefore primarily a formatting/alignment issue rather than an absence of generated output.

2.7 B.4: Out-of-distribution generalization

OOD performance drops sharply: A4 exact micro F1 falls from 94.86% to 17.91%, while leak rate rises from 3.81% to 35.34%. This is mainly a recall failure: exact micro precision remains 22.34%, but recall is only 14.95%. At the same time, over-masking decreases to 0.098%, indicating that the model becomes conservative rather than masking unfamiliar text aggressively.

The OOD split has a substantially different label distribution. Its frequent classes include `FIRSTNAME`, `LASTNAME`, `DATE`, `PREFIX`, `AMOUNT`, `JOBAREA`, `PASSWORD`, `URL`, and `ACCOUNTNUMBER`, rather than the dominant in-distribution classes such as `TIME`, `USERNAME`, and `SOCIALNUMBER`. Thus, the sanitizer generalizes poorly to the shifted entity schema and formatting patterns. Presidio and regex also perform poorly on OOD, but Presidio has a somewhat lower OOD leak rate (41.40%) than regex (85.27%), while A4 remains the strongest of the three systems by both exact F1 and leak rate.

3 Part C: Privacy attacks and privacy–utility trade-off

Evidence and reporting convention. The consolidated measurements are from `results/partC_comparison.json`; the model-level summary is `results/partC_results_table.csv`, and the same-axis canary figure is `results/partC_exposure.pdf` (also available as `.png`). The corrected sanitized targeted evaluation is recorded in `results/partC_attacks_corrected.json`,

Table 5: Preliminary Part C three-victim comparison. Discoverable recovery is the overall exact-match rate; targeted rates are member leak rates (%).

Victim	Held-out PPL	Discoverable	Exposure (bits)	Greedy	Sampled 10
V-base	13.6288	0.00%	1.89	0.5%	1.5%
V-raw	8.9197	35.18%	10.09	4.5%	12.0%
V-san	14.7902	0.66%	1.00	0.5%	1.5%

Table 6: Discoverable exact recovery by repetition count. The denominator is the eligible corpus subset shown in parentheses.

Victim	1 ($n = 3379$)	2 ($n = 98$)	8 ($n = 360$)	32 ($n = 1504$)
V-base	0.00%	0.00%	0.00%	0.00%
V-raw	0.27%	6.12%	100.00%	100.00%
V-san	0.44%	4.08%	4.44%	0.00%

with its execution status in `results/partC_corrections_status.json`. The correction status records that discoverable memorization and canary exposure have not yet been rerun, so the current Part C aggregate values are preliminary and must not be presented as final until all three victims are evaluated with the same corrected attack implementation. Percentages below are rates multiplied by 100. No unredacted PII or generation dump is included in this report.

3.1 C.1 Victim setup and utility

We compare the unfine-tuned Qwen2.5-0.5B model (V-base), a victim trained on the raw corpus (V-raw), and a victim trained on the corresponding sanitizer-output corpus (V-san). V-raw and V-san use the same PEFT recipe: four epochs, batch size 4, gradient accumulation 4, learning rate 2×10^{-4}, seed 42, and LoRA $r = 32$, $\alpha = 64$ on q/k/v/o and gate/up/down projections. The V-san training metadata is retained in `results/partC_vsan_training_metrics.json`. Held-out token-weighted perplexity is 13.6288 (V-base), 8.9197 (V-raw), and 14.7902 (V-san). Thus, sanitization costs 5.8705 PPL points (65.8%) relative to V-raw, and is 1.1614 PPL points above V-base.

3.2 C.2 Discoverable memorization

For each eligible document, the attack continues a fixed prefix and scores an exact continuation match; 2,666 short documents were excluded, leaving 5,341 eligible corpus records. Table 6 shows a sharp repetition effect for V-raw: recovery is 0.27% after one occurrence, 6.12% after two, and 100% at both 8 and 32 repetitions. V-san lowers the overall exact-match rate from 35.18% to 0.66%, a 98.1% relative reduction. The reduction is strong but not absolute: V-san still exactly recovers 4.44% of the eight-repetition group and has a slightly higher one-repetition rate (0.44% versus 0.27%).

3.3 C.3 Untargeted extraction

Not evaluated. The supplied archive did not contain `partC_pii_index.jsonl`; this is explicitly recorded as `pii_index_status: not_provided` in `results/partC_comparison.json` and as `not_evaluated` in `results/partC_corrections_status.json`. Consequently, PII recovery, recovery rate, and the currently stored precision@100 fields are not reported as final results and must not be interpreted as zero recovery. This experiment requires rerunning once a valid index is available.

3.4 C.4 Canary exposure

Figure 2 plots V-raw and V-san on the same axes for 20 canaries (five at each repetition count). Mean V-raw exposure is 10.09 bits, versus 1.00 bits for V-san—a 9.10-bit (90.1%) reduction. By repetition count, V-raw/V-san exposures are 9.58/0.93 (1), 10.65/1.04 (4), 10.25/0.49 (16), and 9.90/1.53 (64) bits. Exposure uses a sampled candidate-rank estimate (10,000 candidates); when exposure is high and ranks concentrate near one, its resolution becomes coarse, so the high-exposure values should not be read as finely resolved rank differences.

3.5 C.5 Targeted PII extraction

Each victim was tested on 200 member and 200 control prompts. A leak means the gold value appeared in the greedy completion, or in at least one of ten sampled completions. Greedy and sampled results are deliberately separated in Table 7; sampling materially increases V-raw member leakage from 4.5% to 12.0%.

Trace status. The retained attack summaries contain aggregate counts rather than per-prompt generations. They therefore cannot establish the two traced leaks required by the assignment. Before the final PDF is produced, a trace-enabled targeted rerun must retain two salted row identifiers, the attack condition, a redacted prompt, a format-preserving redacted recovered value, and exact-match verification. Raw prompts, raw values, and generation dumps must remain out of the report and submission archive.

3.6 C.6 Comparative analysis

Sanitizing the training corpus substantially reduces measured leakage, but trades off held-out utility. Relative to V-raw, V-san reduces member targeted leakage by 88.9% for greedy decoding (4.5% to 0.5%) and 87.5% for sampled decoding (12.0% to 1.5%), while increasing PPL by 65.8%. It also lowers mean canary exposure by 90.1% and discoverable exact recovery by 98.1% overall. Nevertheless, the sanitizer is not a complete privacy guarantee: it permits one greedy and three sampled member leaks among 200 prompts, retains exact recovery for some repeated records, and does not improve the control greedy rate (2.5%) or

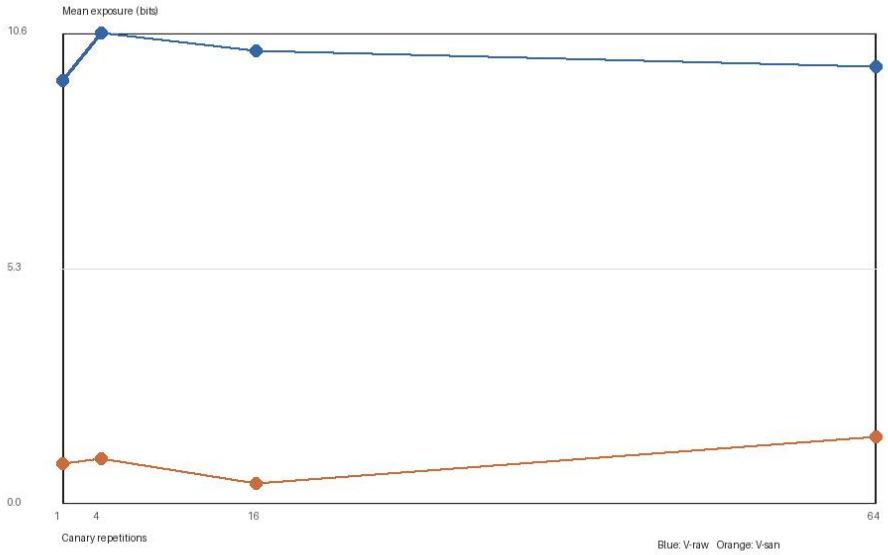


Figure 2: Mean canary exposure against repetition count, with V-raw and V-san on the same axes. BLUE=V-RAW and ORANGE = V-SAN Source: `results/partC_exposure.pdf`.

Table 7: Targeted extraction leak rate (%). Each member/control column has $n = 200$. The V-san rates use the corrected result file.

	Member		Control	
Victim	Greedy	Sampled (any 10)	Greedy	Sampled (any 10)
V-base	0.5	1.5	2.5	6.0
V-raw	4.5	12.0	2.5	4.0
V-san	0.5	1.5	2.5	5.0

sampled control rate (5.0%). The control outcomes are not evidence of training membership by themselves, but they show that prompt and value overlap can still trigger apparent exact matches. These remaining leaks, especially at high repetition, are the concrete sanitizer failures that should be addressed before relying on sanitization alone.
What would you do next? Stronger sanitizer coverage for OOD classes, confidence-based fallback to Presidio/rules, stricter masking validation, repetition limits before victim training, canary audits, and extraction-attack testing before deployment.

Declaration

I have taken help from the following students and LLMs in terms of discussing ideas and coding practices, all my code is written by me.
Consulted students: `<None or names>`.
LLMs/tools used: `Gemini and GPT for Report Formatting, Result nalaysis and Some code-flow analysis`.